

ATTORNEY SHIELD

ASI 2.0 - Native Android & iOS

Attorney Shield 2.0 - What the Native

Colour system: Attorney-Shield Color Scheme Review

Repository: [samson-phillip/asi-claude](#)

DRAFT FOR APPROVAL

Contents

- 1. Dev data is not seeded - this is what blocks us 3**
- 2. Missing from the schema entirely 4**
 - 2.1 Situation preferences - screens 13B, 13C, 27B 4
 - 2.2 Phone capture and verification - screens 08 and 09 4
 - 2.3 Pronouns - screen 10 5
 - 2.4 Trial and guest - screens V1-V2, T1-T8, G1-G3 5
- 3. Shapes we would rather confirm than guess 5**
- 4. Deep-link contract for the web->app handoff 6**
 - Related, and a web task rather than a backend one 6
- 5. One security note 6**
- 6. What we ship as each lands 6**

From: mobile (Android + iOS) **Date:** 2026-08-13

The native apps now sign in against gateway-dev - by password **and by one-time code** - reach the home screen, and complete the registration steps that have endpoints (personal details, address, security PIN). This is the short list of what is still missing, plus a few shapes we would rather confirm than guess.

Where something has no backend we have **left it unbuilt and listed it here**, rather than shipping a placeholder that looks finished.

1. Dev data is not seeded - this is what blocks us

Correction to an earlier version of this document. We previously told you no countries were configured. That was wrong, and the detail matters. Signed in as the test member against <https://gateway-dev.attorneyshield.io/query>:

Query	Result
<code>countries { iso2 }</code>	<code>[]</code>
<code>country(id: "f989d06a-8813-11f1-a446-06cf81ac74a7")</code>	United States , US Yes
<code>subdivisionsByCountry(countryId: <that US id>)</code>	all 50 states + DC Yes
<code>adminIncidentTypeList(activeOnly: false)</code>	<code>[]</code>
<code>adminIncidentTypeList(activeOnly: true, countryISO2: "US")</code>	<code>[]</code>
<code>casesByUser(userID: <test member>)</code>	<code>[]</code>
<code>adminLanguageList</code>	1 entry - ar-SA, isDefault: false

So country and subdivision data **is** there. It is the countries **list query that returns empty** while `country(id:)` and `subdivisionsByCountry` both work on the same records. Our test member's profile even points at that US id.

We have worked around it by reading the country from the member's own profile rather than a picker, which is arguably more correct anyway - but the list query looks broken or scoped in a way nobody intended, and we would rather you knew than have us quietly route around it.

What we still need on dev:

- 1 **countries fixed** - it returns `[]` while the underlying rows resolve fine by id. Possibly a scope/filter issue.
- 2 **Incident types seeded**, with translations. Still `[]` authenticated, with and without a `countryISO2` filter, so this is not a country-filter side-effect. The design reference uses: Traffic Stop, Auto Accident, Pedestrian Stop, Domestic, Test Call, Other. **This is the one that blocks the home screen and the call flow.**

- 3 **An English language entry**, ideally `isDefault: true`. Today the only language is ar-SA and it is not marked default, so even with incident types present our label resolution (English -> org default -> first -> humanized code) would fall through to Arabic or a humanized code.
- 4 **A case for the test member** (munyira851@gmail.com), so jurisdiction and partner resolve from real data instead of falling back to `DEV_DEFAULTS`. The member's organization resolves correctly already (6c53e00d-8682-11f1-a446-06cf81ac74a7), but with no case we cannot exercise attorney pre-selection at all.

The home screen currently shows "No incident types are configured yet.", which is correct behaviour on our side but means we cannot exercise the incident tiles, the attorney chip row, or place a call with a real incident type.

If it is easier to point us at an environment that already has this data, that works too - we only need somewhere to develop against.

2. Missing from the schema entirely

Four areas have no backend, so we have not built them. In each case we have left the gap visible rather than filled it with a placeholder:

2.1 Situation preferences - screens 13B, 13C, 27B

The member's saved "three most common situations". Nothing matching situation, preference or favourite in 238 queries or 297 mutations.

Home currently shows the full incident list, because there is nowhere to store a choice of three. Roughly: a read and a write, capped at three incident-type IDs per member.

2.2 Phone capture and verification - screens 08 and 09

The design's registration completion starts with "Enter your phone number" then "Verify your phone" with a 6-digit texted code. **We have built neither, on purpose.**

- `updateMyContactInfo(input: { phoneE164 })` can store a number. That works.
- **Nothing sends or checks a phone code.** We searched every one of the 297 mutations; there is no `requestPhoneOtp`, no `verifyPhone`, nothing. `requestLoginOtp(channel: SMS)` is a *sign-in* code to an already-verified phone, so it cannot verify a new one.
- `phoneVerifiedAt` is settable only through `CreateUserInput` / `UpdateUserInput` - admin operations, not member self-service.

We could have shipped the entry screen alone, but its own sub-line promises "We'll text a code to verify it", and rewording that to hide the missing half would leave a feature that looks finished and never gets revisited. So both screens are here instead.

What we need: an operation a signed-in member can call to send a code to a new number, and one to verify it - setting `phoneVerifiedAt` on success. Also worth confirming the code length; sign-in codes are 4 digits but the design says 6 for this one.

2.3 Pronouns - screen 10

The design's personal-details screen collects date of birth, gender **and pronouns**. DOB and gender both exist (updateMyProfile). Pronouns do not exist anywhere: no field on any of the 200+ input types, no type with pronoun in the name.

We left the input out rather than collect it and drop it on the floor. One nullable string on UpdateMyProfileInput would do it.

2.4 Trial and guest - screens V1-V2, T1-T8, G1-G3

Nothing matching trial or guest.

The design has a 7-day limited trial with an in-app conversion gate, and a guest mode entered from an unrecognised email at sign-in. **The question that decides the whole design: is a guest a real account with a role, or purely local state?** We would rather ask than assume.

3. Shapes we would rather confirm than guess

Small answers, but each one is currently a guess:

- 1 **countryISO2 on verifyLoginOtp** - we send null, because it is optional and that is what the web client sends. But what is it *for*? If it affects routing or jurisdiction we would rather send something real than a null that quietly degrades.
 - 2 **Sign-in codes are 4 digits** - confirmed against the live gateway, and we have built for 4. Flagging only because the design reference specifies a six-digit entry; we assume the reference is describing registration phone verification, which is a different code.
 - 3 **refreshToken** - what is the access-token lifetime, and does refreshing rotate the refresh token? This matters more for us than for web: a browser tab is short-lived, but a native app sits backgrounded for days, and this is an app people open during a police encounter.
 - 4 **setMemberPin(userId, pin) / verifyMemberPin** - the design reference says the PIN's only job is ending a live session securely; it does not unlock the app or protect recordings. Is verifyMemberPin the intended server-side gate for ending a call?
 - 5 **Casing is inconsistent and we follow whatever each operation uses** - the gateway mixes `userId` (login, casesByUser) with `userid` (setMemberPin, verifyMemberPin), and comms REST uses `memberUserId`. Worth knowing before anyone adds a field.
 - 6 **Is the one-device-at-a-time rule permanent?** LoginPayload returns `otherSessionsRevoked` and `mySessionStatus` reports `another_device`, so signing in on the web ends the app's session and vice versa. We can live with it, but it is worth confirming it is deliberate for a phone people open during a police encounter - if someone signs in on a laptop, the app in their pocket is signed out.
-

4. Deep-link contract for the web->app handoff

Screens 07 and T4 hand the member back to the app "with email pre-filled", but the design reference never records the actual path or parameter names.

Currently implemented: we accept `/app/return`, `/return-to-app` and `/app` on `attorney-shield.com` and `www.attorney-shield.com`, reading an email query parameter. All of it is confined to one file, so a confirmed contract is a one-line change.

We need: the real path and parameter names.

Please do not design the link to carry a credential. We treat it as untrusted input - anyone can send a link. The email is a text-field prefill only, and we have a test asserting that a link carrying `accessToken`, `userId` or `roles` yields nothing but the email. If the app needs to know what someone has bought, we would rather ask the backend after a real sign-in.

Related, and a web task rather than a backend one

For the link to open the app *silently*, both hosts need:

- **Android:** `/.well-known/assetlinks.json` with our signing-certificate SHA-256 fingerprint for `com.app.attorney.shield`
- **iOS:** `/.well-known/apple-app-site-association` for our Team ID + `com.app.attorney.shield`

Until then Android shows a "which app?" dialog and iOS universal links do not fire at all. We will send the fingerprint and Team ID once our release keystore exists.

5. One security note

`POST /api/vonage/video/member-call` **takes no authentication.**

We are not designing around it and nothing we have built depends on it staying open - but as it stands, anyone can place a call against any `organizationId/memberUserId` they can guess, and that call routes to a real attorney.

Flagging rather than assuming it is known.

6. What we ship as each lands

You give us	We ship
Countries, incident types, a language, and a case on dev	Home and the call flow verified against real data
Answers to §3	Token refresh

You give us	We ship
Phone send/verify operations (§2.2)	Registration screens 08 and 09
A pronouns field (§2.3)	The last field of screen 10
Situation-preference endpoints (§2.1)	The home screen's saved three, as designed
A decision on the guest model (§2.2)	Trial and guest flows scoped
The deep-link contract (§4)	A one-line change, then verified

Everything in §3 we can start immediately - the operations are in the schema and the screens are specified in the design reference.